Chapter One

What is artificial intelligence (AI)?

Artificial intelligence (AI) is technology that enables computers and machines to simulate human learning, comprehension, problem solving, decision making, creativity and autonomy.

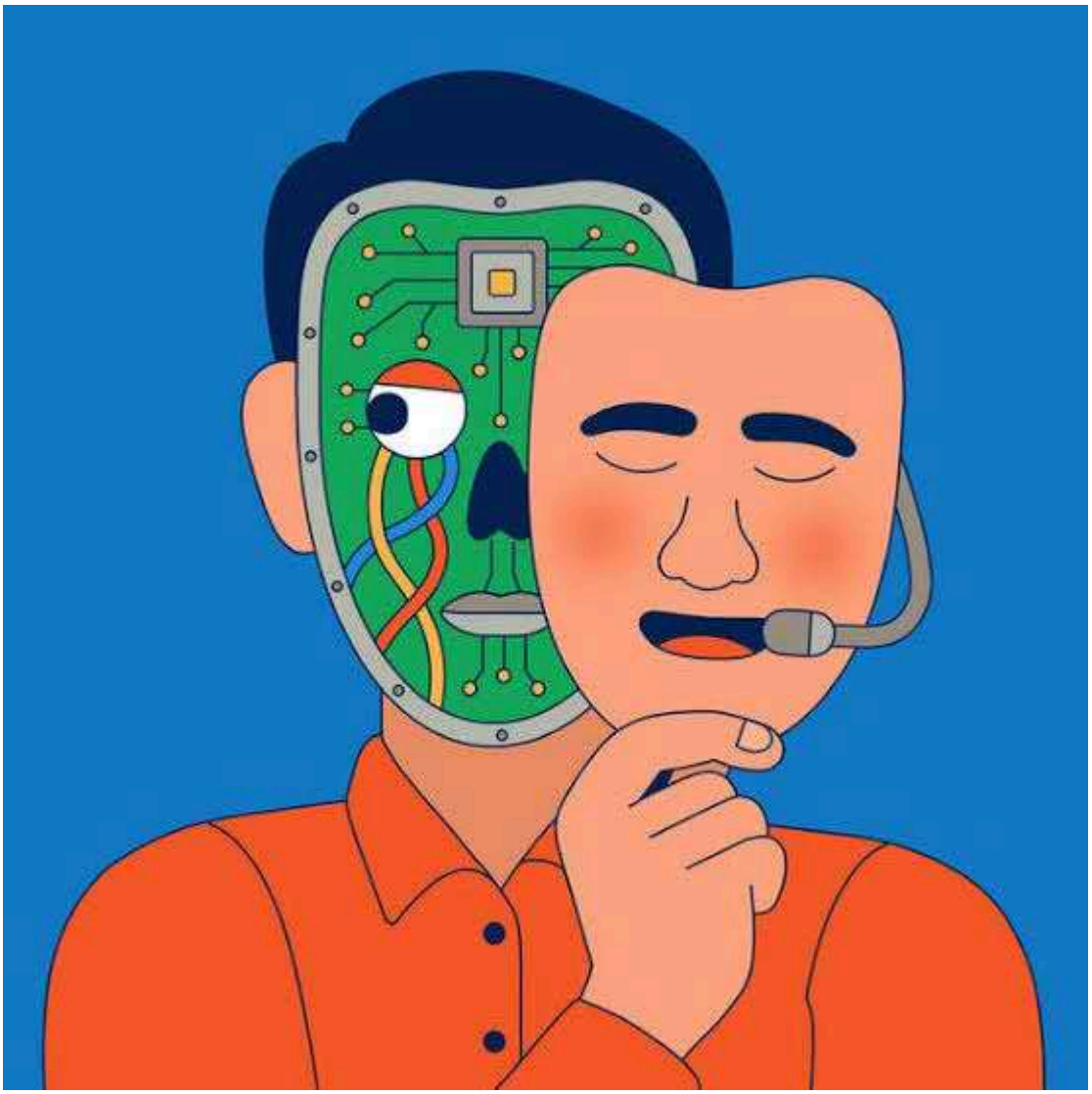
Applications and devices equipped with AI can see and identify objects. They can understand and respond to human language. They can learn from new information and experience. They can make detailed recommendations to users and experts. They can act independently, replacing the need for human intelligence or intervention (a classic example being a self-driving car).

But in 2024, most AI researchers, practitioners and most AI-related headlines are focused on breakthroughs in generative AI (gen AI), a technology that can create original text, images, video and other content. To fully understand generative AI, it’s important to first understand the technologies on which generative AI tools are built: machine learning (ML) and deep learning.

We have all used AI, and have seen how powerful it can be but do we really know what it consists of, below we will try our best to explain that.

AI Technologies Include:

- Autonomous vehicles
- Biometrics
- Chatbots
- Facial and voice recognition
- Image generation
- Digital Assistants
- Generative AI tools
- GPS navigation
- Personalized marketing and many more



Fundamental Components of AI

Data: - AI systems learn and make decisions based on data, and they require large quantities of information to train effectively, especially for machine learning (ML) models.

Algorithms: - Algorithms are the sets of rules AI systems use to process data and make decisions. Machine learning algorithms, for instance, learn and make predictions and decisions without explicit programming.

Computing Power: - AI algorithms often need significant computing resources to process large quantities of data and run complex algorithms.

- As we seen above AI needs data to learn and be capable to make small decisions or give us answers to the prompts we put. There are multiple ways of to train AI, but we will look at the major three ones:-

---

- But first we need to know what training AI actually means; We can compare an AI with a child on how it trains to understand things:-

---

AI	A Child
Given a massive amount of data to make it remember	*Exposed to an environment of people and materials*
Decrease or increase the volume based on performance	*Given positive reinforcement every time it does something good*

- Modern LLMs (large language models), like ChatGPT, use what's called reinforcement learning from human feedback, RLHF, to train a reward model which then is used to train the language model.
- Basically, we attempt to instill an untrained model with weights selected through human preference (which looks more like a cat? which sentence is more polite?). This then automates the process and scales it to superhuman levels which are capable of training massive models like ChatGPT with hopefully something close to what the humans initially intended.
- AI training is the computational process of teaching an AI model to recognize patterns and logic within data. It transforms raw information like text, code, or images into intelligence, enabling the AI to perform tasks it wasn't explicitly programmed for-such as reasoning, problem-solving, and content generation.
- AI models have varied intelligence, but it is not just model specific, it also happens within different versions of a model based and we classify it as:-

1. Narrow AI(Weak AI).

- Weak AI, or Artificial Narrow Intelligence (ANI), refers to those systems that are designed to perform specific tasks or solve particular problems within a defined scope—they cannot think or make decisions beyond them. Unlike humans, who can apply intelligence across different situations, Narrow AI operates under set constraints without general cognitive abilities.
- A great example would be the algorithm that recommends us the videos we watch.

2. General AI(Strong AI),

- Unlike the previous one, this one possesses the ability to think to apply previous information to a new situation it stumbles on.
- The amusing thing is still a concept and hasn't been made yet.

3. Superintelligent AI.

- This introduces a level of intelligence that surpasses human intelligence, and we be able to solve most of everything.
- This is also hypothetical.

- Also we classify AI based on functionalities, as: -

1. Reactive Machines.

- Reactive machine AI refers to the most basic level of artificial intelligence. These systems respond to specific inputs with predetermined outputs without the ability to store data or learn from past experiences. They are designed to react in real time, making them effective for straightforward tasks.
- A great example for these will also be YouTube's search engine.

2. Limited Memory AI.

- Unlike the previous one this model will be able to store data to help it "evolve" or learn.
- Self-driving cars are under this category of AI.

3. Theory of Mind AI

- This is an AI in the works with an aim of to understand human cues and emotions.
- If someone yells at Google Maps out of frustration, it will still deliver the same traffic updates without offering reassurance or support. However, advancements like ChatGPT are beginning to improve in this regard.

4. Self-Aware AI.

- This a stage of AI that we see in movies or read about in books, where an AI model would possess consciousness.

- **There are three stages of AI training where the AI slowly develops: -**

1. Pre - Training Phase.

- Pre training is the initial, large-scale learning phase in which a model is trained on massive, general-purpose datasets-such as text, images, audio, or video-to learn broad patterns and representations. This phase draws heavily on advances in machine learning theory and large-scale distributed systems.
- At this stage the AI gets the basic knowledge it has, such as understanding different languages, recognizing images, and it will be able to identify any relationship between data. This stage looks a lot like how child grows into his/her environment.
- This stage is resource-intensive and only done once to build the foundation or the base.
- At this stage the AI can do most of the things it is suppose to do, and one of maybe the most useful ones is, token-prediction, where the AI predicts the next word or character helping the process be faster.

2. Post - Training Phase.

- Post training is the refining period where we suit the AI for a specific task or domain. At this point the only thing we need to do is fine-tuning where we add restrictions, TOS, and customer service.
- The AI is given human-crafted prompts to tune its conversation capability and safety.

- If pre-training is like teaching a model foundational skills in school, post-training is job training. For example, a large language model can be post-trained to perform sentiment analysis, translation, or understanding of domain-specific language in fields like healthcare, law, or finance.

3. Reinforcement Learning From Human Feedback(RLHF).

- Humans rank different AI responses. A reward model uses these rankings to teach the AI which answers are accurate, helpful and safe.

Chapter Two

- But while doing this there are three methods to train the AI, which are: -

1. Supervised Learning.

- This a machine learning technique where an algorithm is trained from labeled data, like a flashcard that says "This is a person eating corn" matching inputs to known outputs.
- The machine learns mapping first where it maps the input and the outputs using a mathematical expression y=f(x), to map inputs "x" to outputs "y".
- The model compares its predictions against the ground truth labels and adjusts its internal parameters to minimize errors. We call this error adjustment.
- After this training is done the model uses its learned relations ships to predict and produce an output for a new, unseen data. We call this generalization.

- There are two main types of supervised learning: -

1. Classification.

- This predicts categorical discrete labels (male vs female, dog vs cat).
- When there are only two possible classes, we call it binary classification, when there is more than two classes we call it, multi-class classification.
- We use classification in email spam detection, image and object recognition and etcetera.
- As an example of supervised learning, consider an image classification model created to recognize images of vehicles and determine which type of vehicle they are. Such a model can power the CAPTCHA tests many websites use to detect spam bots.

2. Regression.

- Regression is used to understand the relationship between dependent and independent variables. In regression problems, the output is a continuous value, and models attempt to predict the target output.
- So basically its main goal is to predict a value or a quantity.

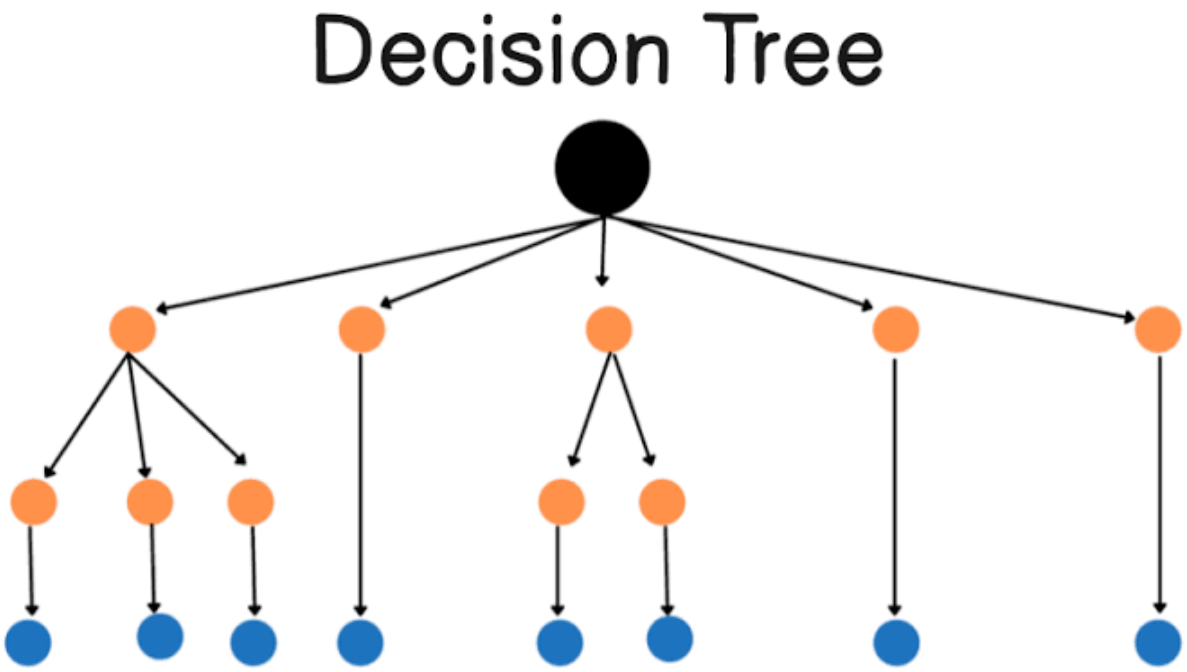
- It predicts and produces and value for a house price and salaries, etcetera.
- Supervised learning uses multiple algorithms, those are : -

1. Linear and Logistic Regressions.

- Linear regression predicts continuous numerical values along a straight line, while logistic regression predicts categorical (binary) probabilities along an S-shaped curve.
    - Linear regression uses a continuous metric output (like price, temperature, or salary).
    - Logistic regression uses a categorical or binary output (like yes/no, pass/fail, or spam/not spam).

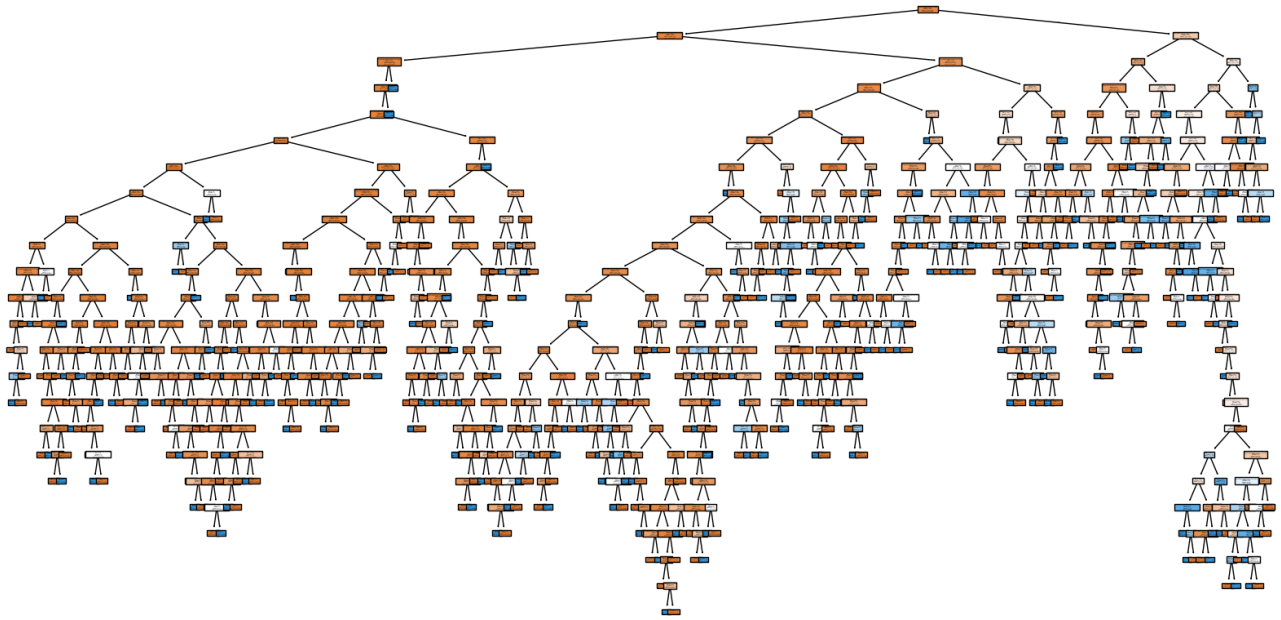
2. Decision Tree and Random Forest.

- **Decision tree** is a single model, while a random forest is an ensemble model made up of many individual decision trees working together.



- Decision tree starts from a single node and splits it to the feature that provides the cleanest separations until it reaches a leaf node which then gives the final prediction. It is easy to visualize and kind of mimics human decision making but it is also prone to overfitting and makes up new data.
- A **Random Forest** fixes the flaws of a single decision tree by creating a "forest" of many independent trees (often hundreds) and aggregating their answers. It relies on the concept that a collective crowd of

average models will outperform a single "expert" model.



- **Bootstrapping (Bagging): -** Each tree is trained on a random sample of the rows from the dataset, sampled with replacement.
- **Feature Randomness: -** When a tree splits a node, it is only allowed to choose from a random subset of features (columns) rather than evaluating all of them.

- Random forest is highly accurate and immune to overfitting but yet so complicated.

3. Support Vector Machines (SVMs).

- It separates different classes of data in an N-dimensional space while maximizing the distance between the classes.

  - **Hyperplane: -** This is the decision boundary that divides the data classes. In a 2D space, the hyperplane is a simple line; in a 3D space, it is a 2D plane; and in higher dimensions, it becomes an N-dimensional hyperplane.

  - **Support Vectors: -** These are the data points that lie closest to the hyperplane. They are the most critical points in the dataset because they define the orientation and position of the decision boundary. If you remove these points, the position of the hyperplane changes.

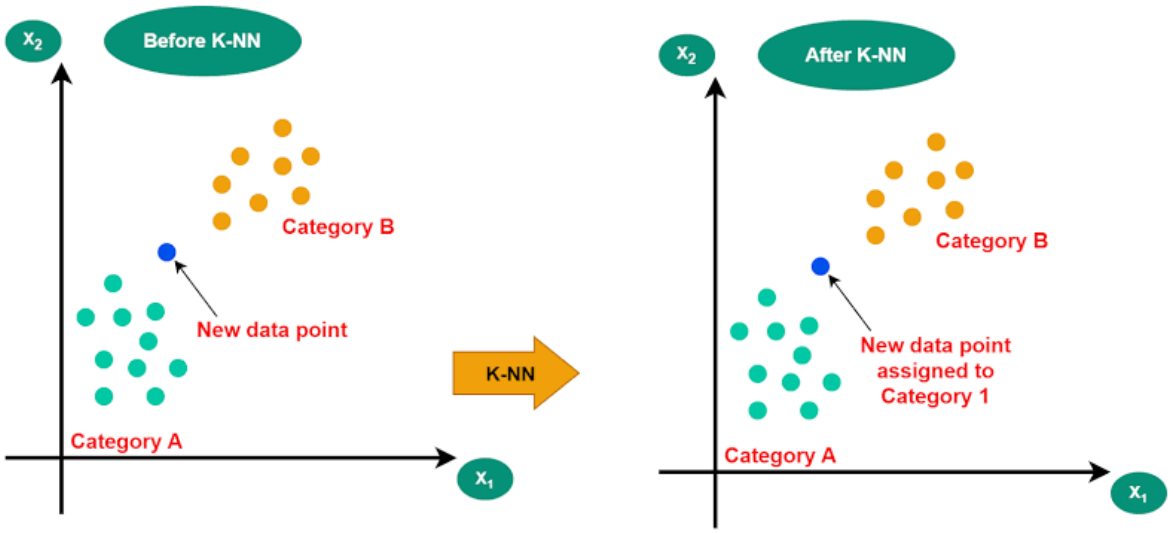
  - **Margin: -** This is the distance between the hyperplane and the nearest data points (the support vectors). SVM aims to achieve a maximum-margin hyperplane, meaning it maximizes this gap to ensure the model generalizes well to new, unseen data.

4. K-Nearest Neighbours.

  - It assumes that similar data points exist close to one another in a feature space.

  - So it measure the distance between the query point and every stored training example using a chosen distance metric.

  - It identifies the k training examples closest to the query point. And after that it will make a prediction.



5. Gradient Boosting.

- It builds a sequential models to fix the errors prior iterations.
- Basically this builds a strong predictive model by leaving the errors and mistakes(pseudo-residuals) of the previous model.

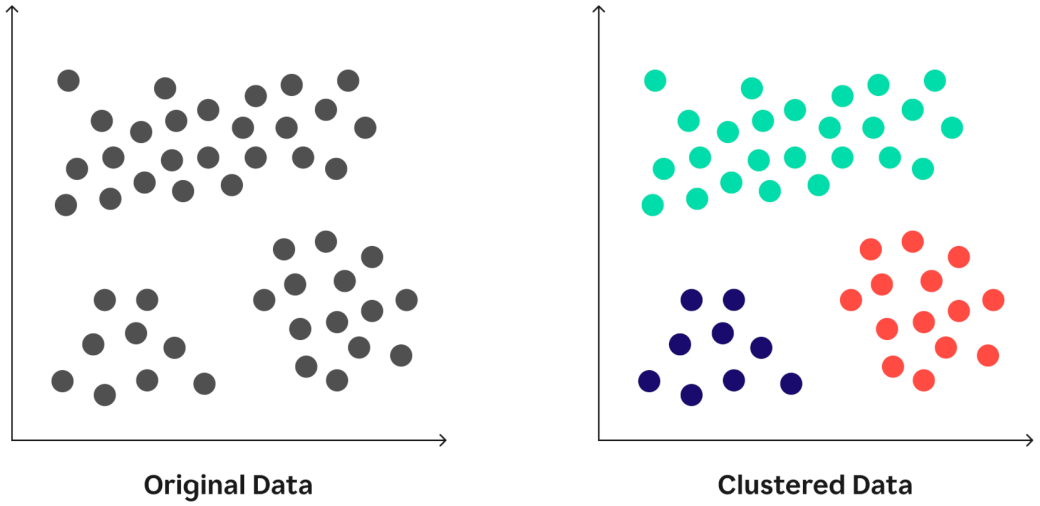
Unsupervised Learning.

- It is a machine learning method where algorithms discover patterns, structures and identify relationships with unlabeled data or predefined outputs without human intervention.
- Unlike supervised learning, in unsupervised learning an algorithm is made to find hidden patterns and relationships based on inherent similarities or even differences.
- The algorithm is given a massive amount of data, without any predefined labels or categories. After that, the algorithm looks for small intricate similarities or hidden patterns within the given set of data so it can match them.
- **There are three algorithms for unsupervised learning.**

1. Clustering.

- Clustering is a technique in unsupervised learning where that groups unlabeled data to groups based on their similarities.

- Its biggest job is to let us understand a data structure for a later use.



2. Association Based Learning.

- It is a technique in unsupervised learning that is rule-based, where it finds interesting relationships between large datasets.
- It identifies patterns in-terms of if-then rules, showing how the presence of one implies the presence of the others.

- So, a person is buying shoes, it is highly likely that the person will buy socks too.

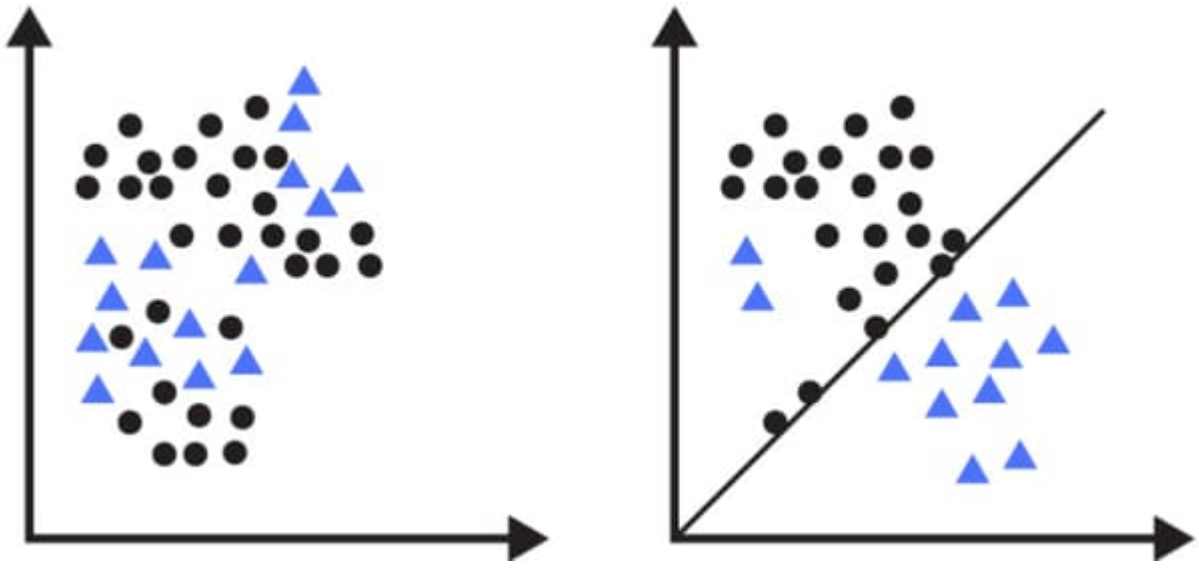

乙万学子的Dream
+关注
平台保障
7天无理由退货·极速退款
林靖昇 北京理工大学良乡校区劲松园C号楼
空白
2本装
(200张共400页)
草稿本
225mm
153mm
实付 ¥6.63 | 优惠前 ¥7.8
共减 ¥1.17 >
1
【空白加厚】5本/共1000页 ¥13.43
【空白加厚】1本/共200页 ¥4.42
近期热销
【空白加厚】2本/共400页 ¥6.63
【空白加厚】10本/共2000页 ¥23.63
【空白加厚】20本/共4000页 ¥47.43
订单备注
无备注 >
顺手买·天猫 好货严选工厂
换一换 >
【8支】0.5mm中性笔
【8支】中性笔;0.5+mm;8支;
退货宝 | 已售6万+



- The image above shows an example of the application of association based learning in an app. In this case, **Taobao** creates a system that gives the user relevant items based on the one they are interested in.
- This is how the model is trained it sees there in an interest in a notebook and if there is an interest in the notebook there must be an interest in a pen too.

3. Dimensionality Reduction.

- It is a process of decreasing the number of variables in a dataset while still retaining the original information. This method avoids redundancy and speedup computation time.
- It is really helpful in spotting suspicious actions like fraud and the similar by spotting the odd one out of a dataset.
- It is also really helpful in customer segmentation, where the algorithm classifies the customer base through their demographics and purchasing habits.



- **Unsupervised learning is really useful when it comes to saving time but also really challenging since the model can pickup noises rather than real data from the dataset leading to overfitting.**

Chapter 3

Machine Learning.

- A simple way to think about AI is as a series of nested or derivative concepts that have emerged over more than 70 years:
- Directly underneath AI, we have machine learning, which involves creating models by training an algorithm to make predictions or decisions based on data. It encompasses a broad range of techniques that enable computers to learn from and make inferences based on data without being explicitly programmed for specific tasks.
- The central premise of machine learning (ML) is that if you optimize a model's performance on a dataset of tasks that adequately resemble the real-world problems it will be used for—through a process called model training—the model can make accurate predictions on the new data it sees in its ultimate use case.
- Nowadays many AI models are used for extensive kind of things from really useful tasks to weirdly as "companions", this is where ML comes in handy.
- Training itself is simply a means to an end: generalization, the translation of strong performance on training data to useful results in real-world scenarios, is the fundamental goal of machine learning. In essence, a trained model is applying patterns it learned from training data to infer the correct output for a real-world task: the deployment of an AI model is therefore called AI inference.
- **AI Inference: -** Is the operational phase where an AI model processes a new unseen data to generate real time predictions, classifications or texts. This is explained extensively above.
- Machine learning is also associated with several other artificial intelligence subfields: -

Deep Learning.

- It is a specialized subset of machine learning that uses multilayered artificial neural networks to automatically learn complex patterns and representations from raw, unstructured data.
- The term **"Deep"** refers to the multitude of hidden layers used, ranging from three to hundreds to thousands that allow computers to solve intricate problems without manual feature engineering.

Natural Language Processing.

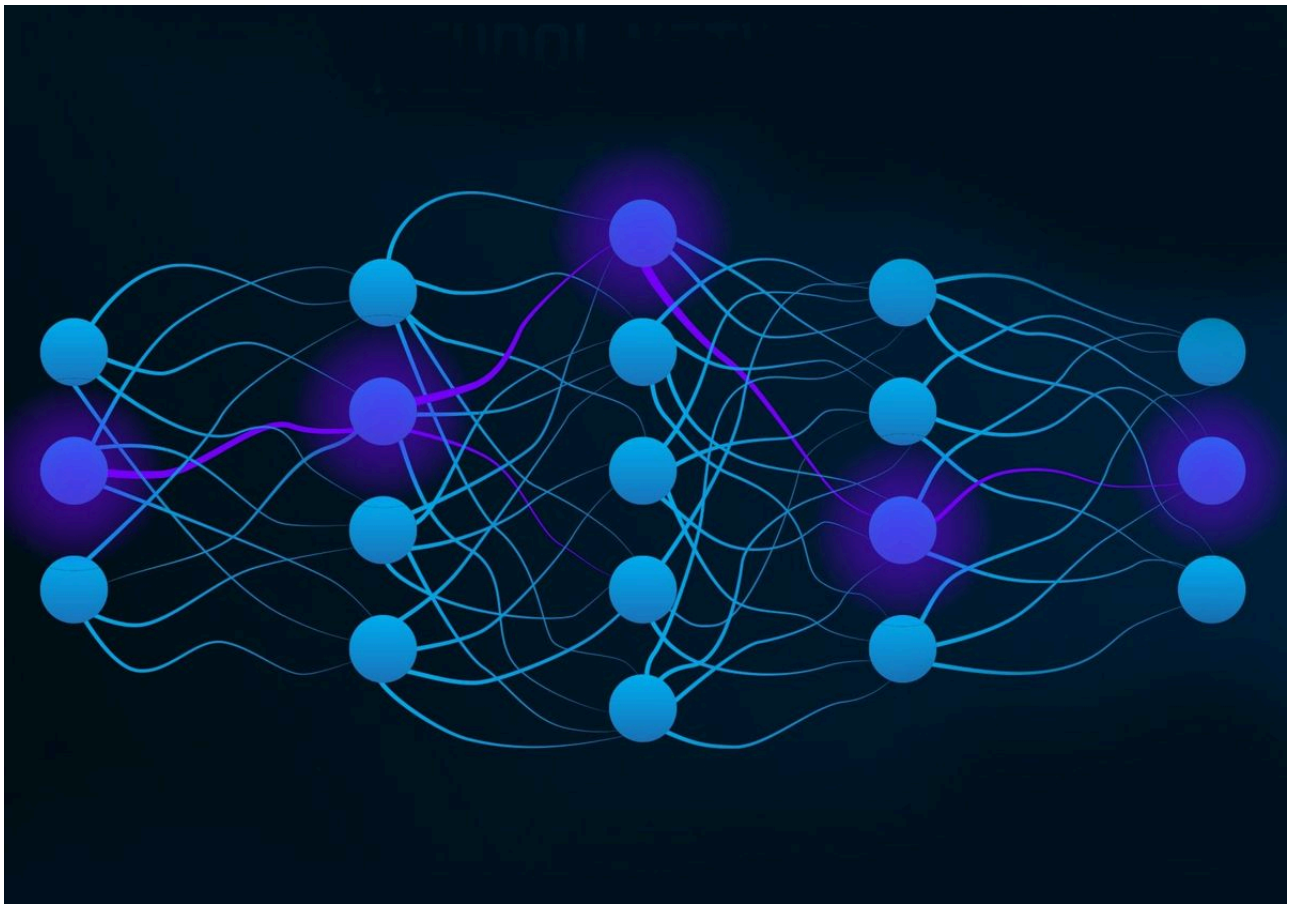
- Natural language processing is a field of machine learning in which machines learn to understand natural language as spoken and written by humans, instead of the data and numbers normally used to program computers. This allows machines to recognize language, understand it, and respond to it, as well as create new text and translate between languages. Natural language processing enables familiar

technology like Chatbots and digital assistants like Siri or Alexa.

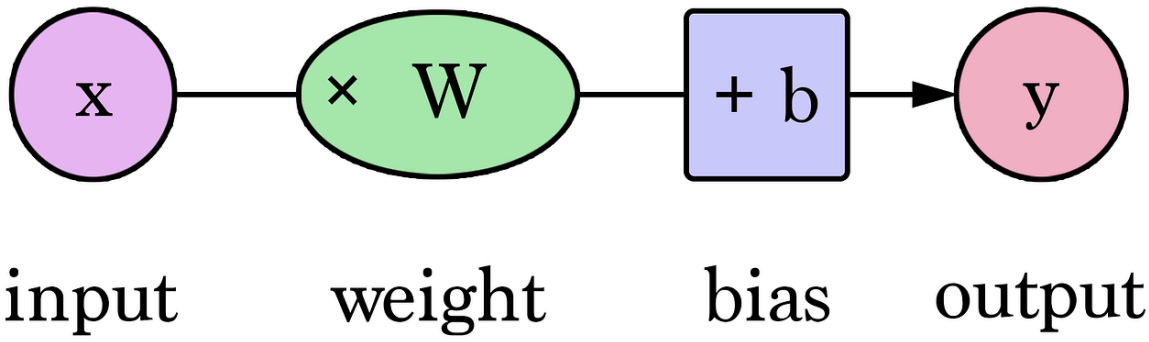


Neural Network.

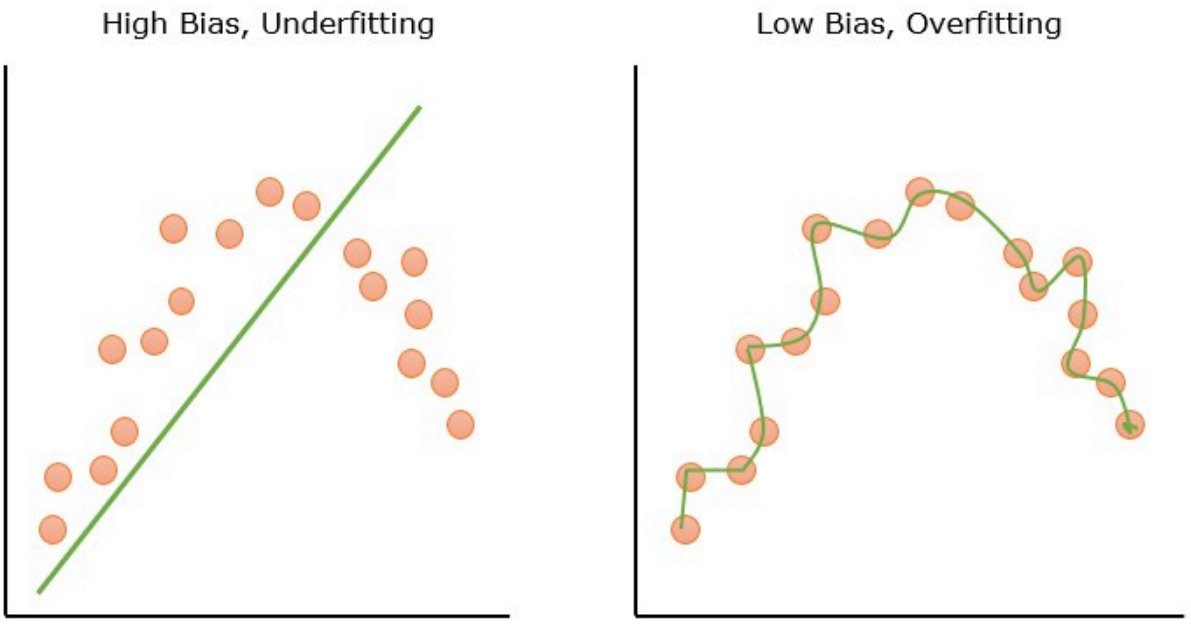
- A neural network is a machine learning model that stacks simple "neurons" in layers and learns pattern-recognizing weights and biases from data to map inputs to outputs.



- While today's deep neural networks (DNNs) power systems as complex as transformers and convolution neural networks (CNNs), the origins of neural networks trace back to simple models such as linear regression and how the human brain digests, processes and decides on the information presented to it.
- On a high level, the inspiration for neural networks comes from the biological neurons in the human brain, which communicate through electrical signals.
- A neural network can be understood through a simple example: spam detection. An email is fed into the network, and features such as words or phrases like "prize," "money," "dear" or "win" are used as inputs. The early neurons in the network process the importance of each signal, while later layers combine this information into higher-level cues that capture context and tone.
- In essence, the network learns how to transform raw features into meaningful patterns and use them to make predictions. This process is powered by two fundamental concepts: weights and biases.
- **Weights: -** Weights act like dials that control how strongly each input feature influences the decision—a word like "prize" may be given more weight than a common word like "hello."



- **Biases: -** These are built-in values that influence the decision threshold allowing a neuron to activate even if the inputs are weak.



- Together, these model parameters determine how each neuron contributes to the overall computation. By adjusting these values during training, the network gradually learns to make accurate predictions. For example, it will be able to determine if an email is really a phishing email or a legitimate one, that why sometimes even legitimate emails end up the spam or junk folder.
- A neural network is composed of: -

Input Layer: - This holds the raw features.

Hidden Layers: - Consists of artificial neurons (or nodes) that transform inputs into new representations. This is where the linear transformation between input and output happens.

Output Layer: - After performing the linear transformation in the hidden layer, a final prediction is made.

- A neural network has a "neuron" for each input (each of the pixels of the images) and a neuron for each output (each of the 10 possible digits).
- A basic neural network just connects each input neuron to each output neuron, and so each input effectively votes directly for which output it thinks is right. For recognizing digits, a pixel right in the center being "on" could be a lot of things, but is almost certainly not a 0.
- So, the connection between the pixel and the zero is weighted low, and is weighted something higher for the rest of the outputs.
- The real beauty of machine learning is that we let it figure stuff out and if it correctly guesses or predicts, we give it some kind of reinforcement, slightly increasing those weighted votes, and if it gets it wrong we correct it so it can lower those weighted votes.

- The different thing about Neural Network is, we can actually ramp it up, unlike the other learning approaches, by increasing the number of hidden layers which basically means adding more computing resources to it.
- The downside lies on the complexity of the result they give us, it is a complex combination of numbers which can lead a person to solve a problem they don't understand, with a solution that they don't understand.
- But for a model to learn from data it need three core steps: -

1. Forward Propagation

- Data flows forward from the input layer, through hidden layers, to the output layer.
- So first the model performs **Linear Combination**, where each neuron calculates a weighted sums of the inputs and adds the bias.
- Then it performs **Activation Function**, where a non-linear function like the ReLU, Sigmoid or Tanh are added to the sum to introduce non-linearity and generate the neuron's output (activation).

ReLu	Sigmoid	Tanh
Outputs the input directly if it is positive, if not the output is zero.	Compresses real valued numbers into probability-like scale between 0 and 1.	Concentrated the value around zero in an shaped curve so it will be easier for weight updates, since the convergence will be around zero.

- Lastly, it gives us a **Final Output**, where the process repeats again and again until the final layer produces a prediction.

2. Loss Calculation.

- This is the stage where error measurement is performed. At this stage the model measures compares its predicted output(let's call it y') to the true target output(y) using a loss function or cost function to quantify its deviation from the true output or how wrong it is.
- The entire training objective is to minimize this loss value.

3. Backpropagation.

- At thi stage what happens is backpropagation moves backwards to see how much each weight and bias contributed to the error.
- This pinpoints where it got it wrong so it will be easier to minimize it on the next rodeo.

Types Of Neural Networks.

- **Convolutional neural networks (CNNs or convents): -** Designed for grid-like data such as images. CNNs excel at image recognition, computer vision and facial recognition thanks to convolutional filters that detect spatial hierarchies of features.

- **Recurrent neural networks (RNNs): -** Incorporate feedback loops that allow information to persist across time steps. RNNs are well-suited for speech recognition, time series forecasting and sequential data.
- **Transformers: -** A modern architecture that replaced RNNs for many sequence tasks. Transformers leverage attention mechanisms to capture dependencies in natural language processing (NLP) and power state-of-the-art models like GPT.
- Regardless of architecture, all of the above rely on core principles: - *Artificial Neurons, nonlinear activations and optimization algorithms.*

How Deep Learning Works.

- We have see the Artificial Neural Networks(ANNs) above but lets expand on the others too: -
- **Hierarchial Feature Learning: -** The early layers detect the smaller details like edges and pixels, the middle layers combine those to make shapes or textures, and deep layers recognize high-level concepts like faces and objects.
- As we mentioned above models also train via backpropagation which is really useful in detecting mistakes by literally back-tracking and trying to pinpoint them.
- But one of teach and money savy operation is: - **Hardware Acceleration,** it is basically using massive amounts of specialized GPUs and processors to perform tasks that are repetitive and making the process of the neural network faster than standard CPUs.
- This process is really important in speeding up processes like forward propagation, loss calculation and backproagation since they need to be done million, if not billions of times.
- Now lets get to the fun part, have you ever imagined how an AI models approaches human intelligence. It is all about mimicking the human brain but not about replicating it, so there must be a way or two or even three to mimic human intelligence, however great it is. The next part walks us through one of the approaches a model mimic human intelligence.

Chapter 4

Connectionist AI: Learning Patterns from Examples

Where this chapter fits. In the Algorithms and Data Structures (DSA) chapter we learned how to **organize** case information and **search** it efficiently. This chapter asks a different question: what happens when a computer **finds a pattern** in that information, and how should an analyst treat the result?

> **Central question:** How can analysts organize case information, search it efficiently, and assess patterns identified by AI without mistaking predictions for verified evidence?

Contents

1. Chapter Introduction: The Harbor-17 Images

2. What Is Connectionist AI?
3. Artificial Neurons
4. Neural Network Structure
5. Training Versus Inference
6. How Data Structures Connect to Neural Networks
7. Prediction Errors and Limitations
8. Activity B: Evaluate an AI Lead
9. End-of-Chapter Review
10. References

---

1. Chapter Introduction: The Harbor-17 Images

In a fictional FBI field office, an analyst receives an AI-prioritized image list and must decide what its scores actually mean.

Imagine a fictional case called **Harbor-17**. Analysts have collected 40,000 photographs from roadside cameras near an invented warehouse district. Someone has asked: "*Which of these images show a truck?*"

In the DSA chapter, we would first organize the images, for example by storing each record in a hash table keyed by image ID so that any one image can be found quickly. But organizing 40,000 images does not tell us what is in them. Reviewing all 40,000 manually would be slow, so a classifier could help **prioritize** images for human examination.

One option is to write rules by hand: "*If the vehicle is longer than 6 metres and has a boxy rear, call it a truck.*" This works for some vehicles and fails for others. Pickup trucks are short, and delivery vans are boxy. Real vehicles keep breaking our rules.

Another option is to train a computer on many images labeled **"truck"** or **"not truck"** and let it **learn useful patterns from examples**, rather than specifying every decision rule by hand. This is the idea behind **connectionist AI**.

This approach is powerful, but it can be oversold. A program that says **"truck, confidence 0.88"** has produced a prediction, not a fact. The rest of this chapter explains how such predictions are produced, why they can be wrong, and why a human must verify them.

Learning goals. After this chapter you should be able to explain:

- what connectionist AI is and how it differs from fixed-rule programs
- how artificial neurons and layers produce a prediction, including one small calculation
- the difference between training and inference
- how arrays and matrices store inputs and weights
- what false positives and false negatives are, and how poor training data causes errors
- why a prediction is a lead to be checked, never established evidence

---

2. What Is Connectionist AI?

Connectionist AI is an approach in which a system learns useful patterns using many simple numerical units, called **artificial neurons**, that are connected together. Much of what the system learns is represented by

numerical **weights** and other parameters, rather than a complete set of hand-written classification rules.

The approach is **inspired** by the brain, in which many simple cells pass signals to one another. It is only loosely inspired. An artificial neuron is a small piece of arithmetic, not a model of a living cell (see Section 3).

Fixed rules versus learned patterns

Aspect	Fixed-rule program	Connectionist (neural network) program
Where the decision criteria come from	A programmer specifies the rules	The network adjusts parameters using examples during training
Harbor-17 example	`if length > 6 m and boxy_rear: "truck"`	A network trained on thousands of labeled images
Strength	Easy to read and explain	Can capture patterns too complicated to write as rules
Limitation	Can fail on cases the author did not foresee	Can be hard to interpret and can fail on data unlike its training examples

A neural network still uses algorithms

Do not think of a neural network as magic. Strictly speaking, the **neural network is a model**; algorithms are the procedures used to train it and calculate its outputs. Those procedures follow defined steps: multiply, add, apply an activation function, and pass values onward. A programmer specifies the model design and computation steps, while training determines many of the numerical **parameters** (such as weights).

> **Check yourself.** A colleague says, **"Neural networks do not use algorithms because they learn their own rules." Answer:** Incorrect. Algorithms perform training and prediction. The network learns numerical parameter values through training; it does not invent its own computing instructions.

---

3. Artificial Neurons

The parts of a neuron

An **artificial neuron** takes in numbers, combines them, and produces one number. Its parts are:

- **Input / feature:** a number describing something about the example. For a vehicle image, a feature might be "how box-shaped is the rear? (0 to 1)". We use two made-up, pre-measured features for easy arithmetic; practical image models can also learn useful features from pixel data.
- **Weight:** a number controlling how much an input contributes. In our simple yes/no neuron, a positive weight **tends to push the output toward 1** when its input is positive (with other values held fixed), while a negative weight tends to push against that outcome.
- **Weighted sum:** multiply each input by its weight and add the results.
- **Activation:** a simple rule that turns the weighted sum into the neuron's **output**. Here we use a **threshold**: output 1 if the sum reaches the threshold, otherwise 0.

A worked example

A single neuron decides "truck-like (1) or not truck-like (0)". It has two inputs:

- `x1` = box-shaped rear score (0 to 1)
- `x2` = relative size score (0 to 1)

The weights are `w1 = 2` and `w2 = 1`, and the threshold is `1.5`. (All numbers are invented.)

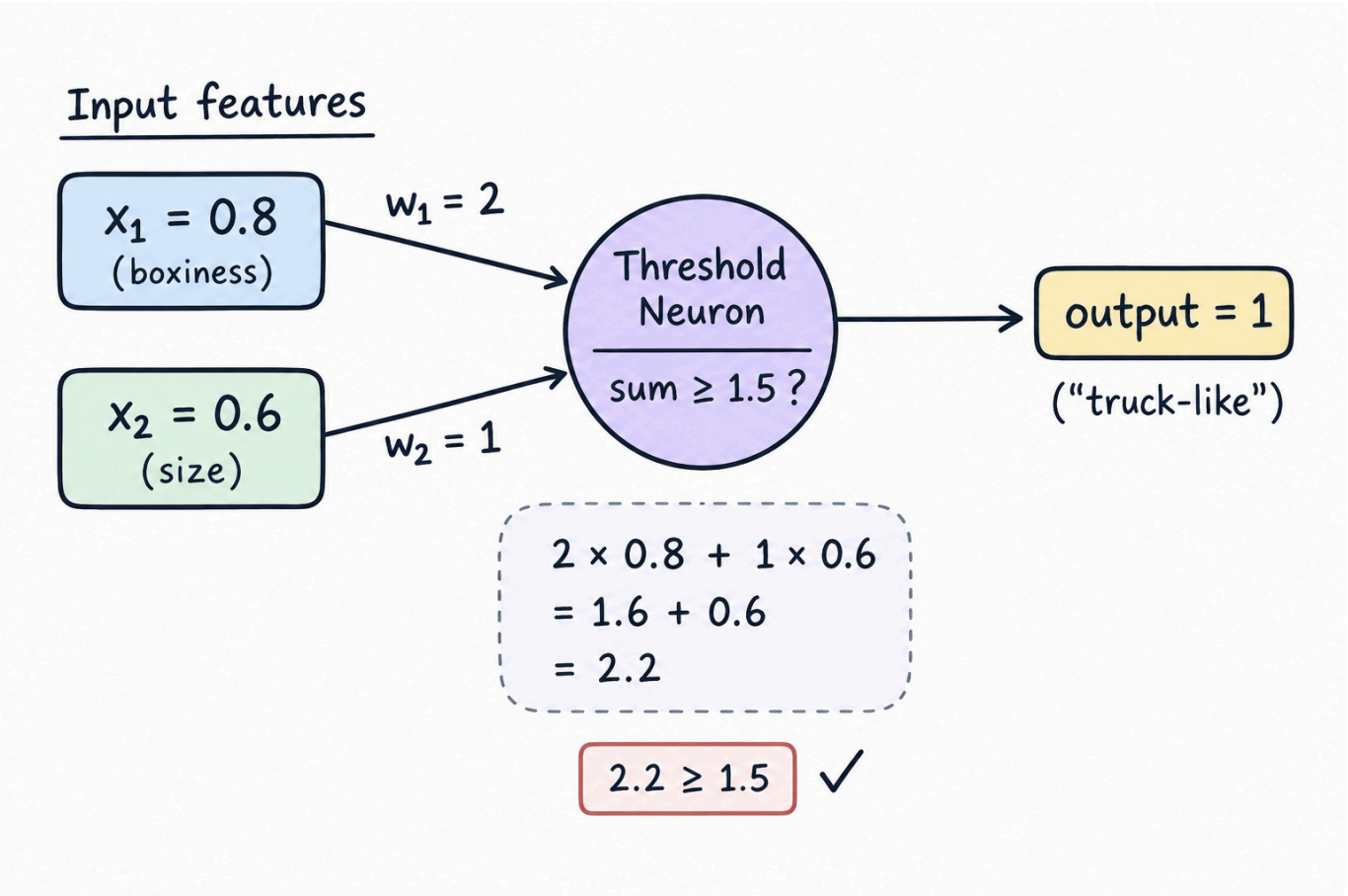
Image A: `x1 = 0.8`, `x2 = 0.6`

```
weighted sum = (2 x 0.8) + (1 x 0.6) = 1.6 + 0.6 = 2.2
2.2 >= 1.5  ->  output 1 ("truck-like")
```

Image B: `x1 = 0.2`, `x2 = 0.3`

```
weighted sum = (2 x 0.2) + (1 x 0.3) = 0.4 + 0.3 = 0.7
0.7 < 1.5   ->  output 0 ("not truck-like")
```

The neuron gave more influence to the boxy-rear feature (weight 2) than to size (weight 1). Changing the weights changes the behavior, and as we will see, *learning means finding good weights*.



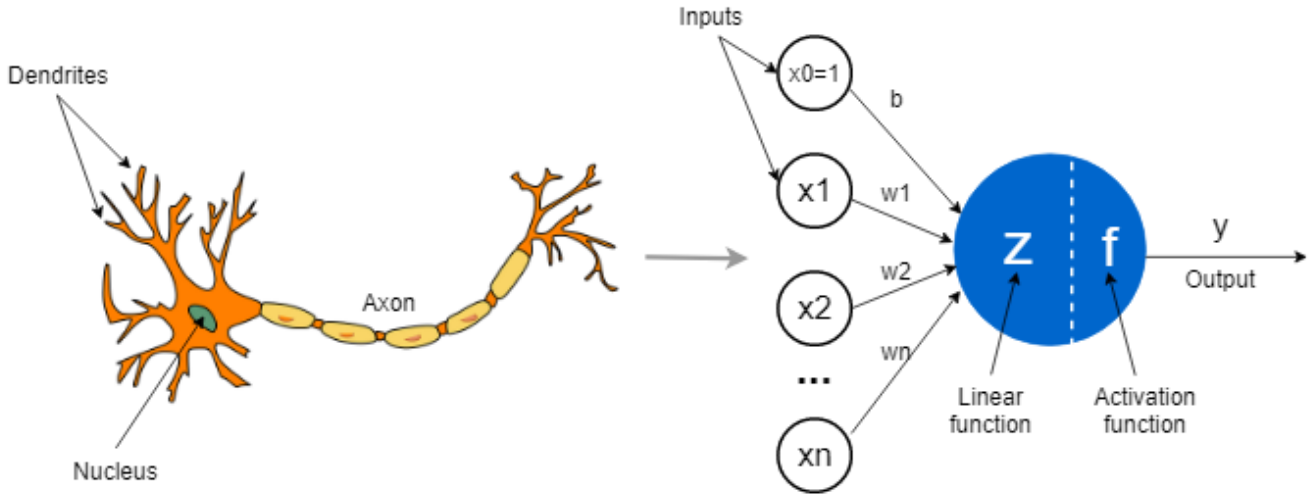
A note on bias

Many descriptions rewrite the threshold as an extra number called the **bias parameter**: "add the bias to the sum, then check whether the result is above 0". Our threshold of `1.5` is equivalent to a bias of `-1.5` when the

rule is **output 1 if** `weighted sum + bias >= 0`. Keep this in mind, because in Section 7 we meet a completely different use of the word **"bias"**.

Neuron versus biological neuron

A **biological neuron** is a living cell with chemistry and structure far richer than a weighted sum. The artificial neuron borrows one *idea* (many signals combine, and the result is passed on) and turns it into arithmetic. Calling it "a tiny brain cell" is a metaphor, not a description.



> **Check yourself.** In the example, what happens to Image A's output if `w1` is changed from 2 to 0.5?
> **Answer:** The sum becomes (0.5 x 0.8) + (1 x 0.6) = 1.0, which is below 1.5, so the output becomes 0. A weight change flipped the decision.

---

4. Neural Network Structure

One neuron can only make simple decisions. A **neural network** connects many neurons in **layers**:

- **Input layer:** receives the numbers describing the example and passes them into the model. In our simple diagram, it does not perform a weighted-neuron calculation.
- **Hidden layer(s):** neurons that take the previous layer's outputs as their inputs. They are "hidden" because they are neither the data we supply nor the answer we read.
- **Output layer:** produces the final prediction.

Information flows in one direction, from input to output, which is called a **forward pass**.

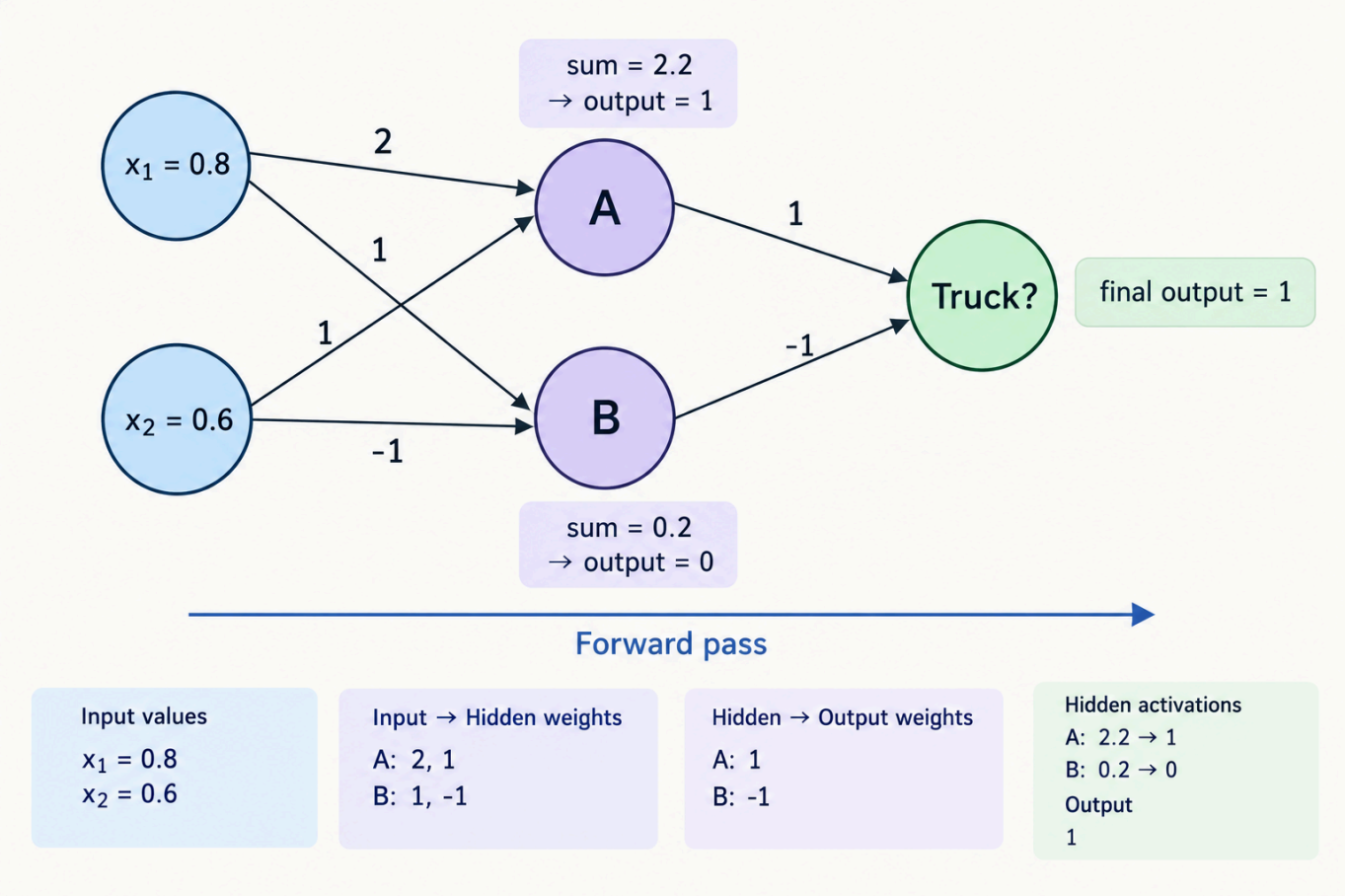
A small illustrative network

We reuse Image A (`x1 = 0.8`, `x2 = 0.6`) and add a hidden layer of two neurons and one output neuron. The labels are invented for teaching.

Neuron	Role (invented)	Weights	Threshold	Weighted sum	Output
Hidden A	"weighted boxiness-and-size signal"	2, 1	1.5	2.2	1
Hidden B	"boxiness-minus-size signal"	1, -1	0.5	0.8 - 0.6 = 0.2	0

Neuron	Role (invented)	Weights	Threshold	Weighted sum	Output
Output	"truck"	1, -1	0.5	(1 x 1) + (-1 x 0) = 1	1

The output neuron receives the **outputs of the hidden neurons**, not the original input numbers directly. For Image A, Hidden A outputs 1 and Hidden B outputs 0, so the final output is 1 ("truck-like"). Importantly, **Hidden A is not an exact "large and boxy" rule**: its first input alone can be high enough to pass the threshold. Its descriptive label is a teaching aid, not a claim about a precise logical condition.



A second forward pass: when both hidden neurons activate

Consider **Image C**, with invented features `x1 = 0.8` and `x2 = 0.2`. Keep exactly the same weights and thresholds.

Stage	Simple arithmetic	Output
Hidden A	`(2 × 0.8) + (1 × 0.2) = 1.8`, reaching `1.5`	`1`
Hidden B	`(1 × 0.8) + (-1 × 0.2) = 0.6`, reaching `0.5`	`1`
Output neuron	`(1 × 1) + (-1 × 1) = 0`, below `0.5`	`0` ("not truck-like")

Both hidden neurons now output `1`. The output neuron's negative weight for Hidden B **counteracts** Hidden A, changing the final prediction from Image A's `1` to Image C's `0`. This is a teaching-only example, not a reliable vehicle classifier: it simply demonstrates why the output layer needs information from **both** hidden neurons.

From toy 0/1 outputs to real-valued scores

Our hand-calculated neurons use a **hard threshold** and output only 0 or 1. Many practical classifiers instead produce a **numerical score** from which a predicted label can be chosen. The scores such as `0.88` in Activity B are invented outputs from a different, more realistic style of classifier—not numbers produced by our toy threshold network. A score is not automatically a trustworthy probability (see Section 7).

Why layers help, and what they do not promise

Layers allow the network to build **combinations** of features. Hidden A gives more weight to boxiness than size; the output layer combines the hidden neurons' results. In real networks, hidden neurons usually do not have tidy, human-readable meanings like the simplified labels above. We invented those names to make the arithmetic understandable. What a real network learns must be investigated, not assumed.

> **Check yourself.** Which layer's output do we read as the network's prediction? **Answer:** The output layer.

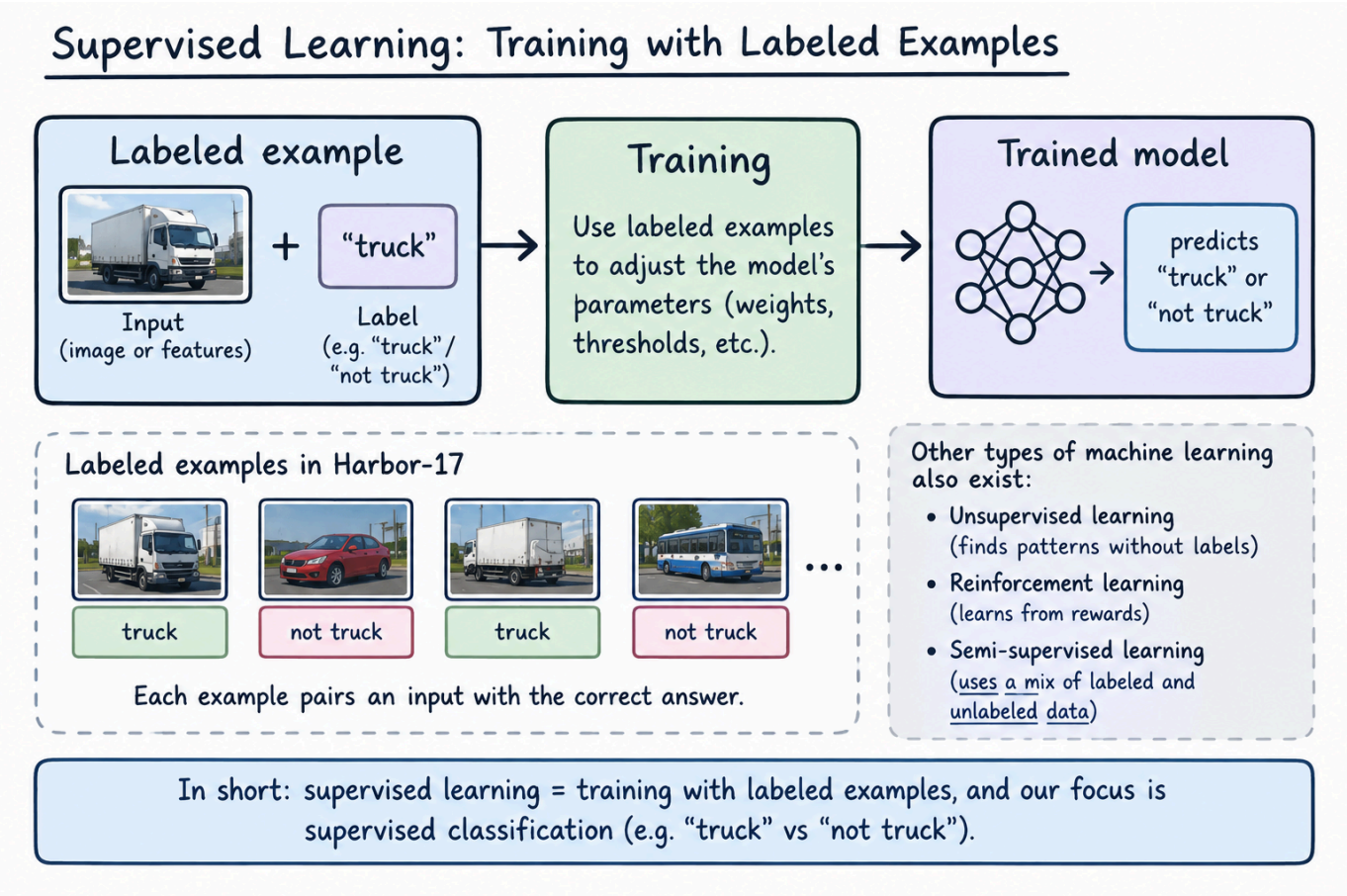
---

5. Training Versus Inference

Training and **inference** are two distinct activities. Understanding which one changes the model is essential.

Training: learning the weights

In our **supervised learning** example, **training** means adjusting model parameters using **labeled examples**. A **labeled example** pairs an input with a reference answer, often supplied by a human reviewer. In Harbor-17, that is an image or its numerical features paired with "truck" or "not truck". Other types of machine learning exist, but supervised classification is our focus.



The main idea is a repeating loop:

1. **Predict:** Feed a labeled training example into the network and get its current guess.
2. **Compare:** Check the guess against the example's reference label.
3. **Adjust:** Use that error as feedback to change the weights in an attempt to improve future predictions.
4. **Repeat:** Practice on many examples, not just one image.

Remember: Training aims to improve performance across different examples. Correcting one prediction does not guarantee improvement on every other image.

Return to our one neuron (`w1 = 2`, `w2 = 1`, threshold `1.5`). A fictional delivery van has features `x1 = 0.9`, `x2 = 0.4` and reference label **0 (not truck)**.

```
Predict: (2 × 0.9) + (1 × 0.4) = 2.2 -> output 1 ("truck")
Compare: predicted 1, reference label 0 -> error
```

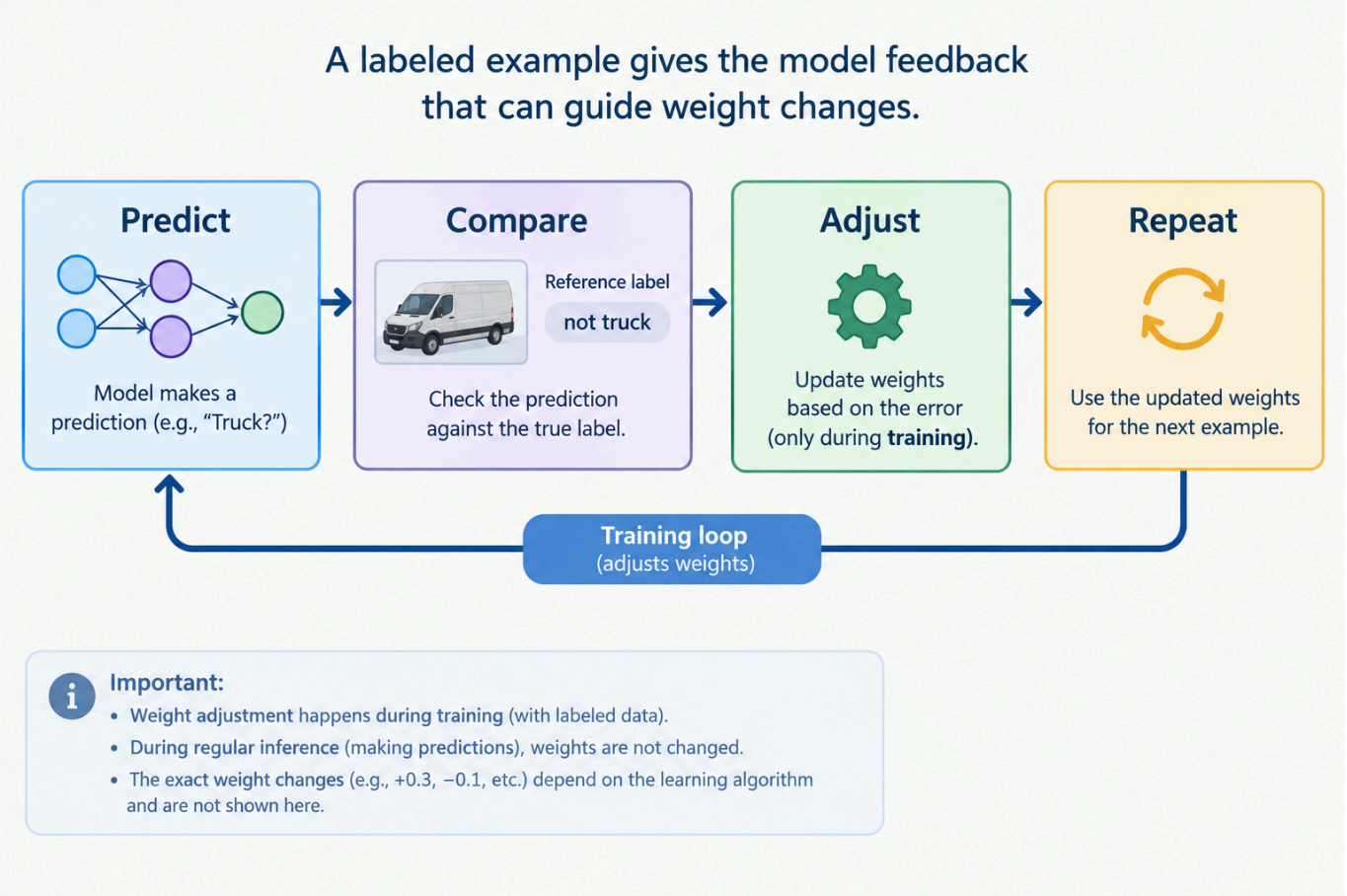
For this **toy perceptron-style example only**, define `error = predicted label - correct label` and use:

`new weight = old weight - (adjustment size × error × input)`

With adjustment size `1` and error `1`:

```
new w1 = 2 - (1 × 1 × 0.9) = 1.1
new w2 = 1 - (1 × 1 × 0.4) = 0.6
Re-check: (1.1 × 0.9) + (0.6 × 0.4) = 1.23 < 1.5
New output: 0 ("not truck")
```

That change fixes the van example. But if Image A is truly a truck, its new sum is `(1.1 × 0.8) + (0.6 × 0.6) = 1.24`, below the threshold. The model now misses Image A. Fixing one example can create another error. Real multilayer networks use more sophisticated learning methods; this optional calculation is only a simple illustration of feedback and adjustment.



Inference: using the learned weights

Inference means using *already trained* parameters to predict on **new inputs**. The model does not need a reference label to make a prediction, and its weights are not updated during ordinary inference.

Suppose training ended with `w1 = 1.8`, `w2 = 1.2` (invented). A new Harbor-17 image arrives with `x1 = 0.7`, `x2 = 0.9`:

```
(1.8 x 0.7) + (1.2 x 0.9) = 1.26 + 1.08 = 2.34  ->  2.34 >= 1.5  ->  output 1
("truck")
```

The network never learned the right answer for this image. It is applying patterns learned from *other* images.

Comparison

Aspect	Training	Inference
Purpose	Find good weights	Use the weights to make a prediction
Data needed	Training examples with reference labels in this supervised example	Inputs to classify; labels are not needed to produce predictions
Are weights changed?	Yes, as training proceeds	No, during ordinary inference

Aspect	Training	Inference
Harbor-17 example	Learn from 5,000 analyst-labeled images	Classify the other 35,000 images
Cost	Usually the slow, expensive stage	Usually fast

Checking whether training worked

The aim of training is not simply to remember the images already seen, but to **generalize:** make useful predictions on new images. A model may perform well on its training examples and still fail on unfamiliar ones; this is one form of **overfitting**.

To check performance, keep some **held-out examples** out of the weight-adjustment process and evaluate the trained model on them. In our fictional case, images from different lighting conditions and camera angles help test whether a daytime-only training set will transfer to new conditions. The eight-image table in Activity B is a **teaching exercise**, not enough evidence to estimate how well a real classifier would perform across thousands of new images.

> **Check yourself.** Why is checking a model only on the pictures it trained on insufficient?
> **Answer:** It may have learned patterns specific to those examples that do not hold for new images. Independent examples reveal whether it generalizes.

Common misconception

> **Misconception:** "The network keeps learning every time I give it a new image." **Reality:** In ordinary inference, the parameters stay fixed. Updating them requires a separate learning or retraining process.

> **Check yourself.** An analyst feeds an unlabeled image into a finished classifier and gets "truck". Was this training or inference? **Answer:** Inference. There was no label, and no weights changed.

---

6. How Data Structures Connect to Neural Networks

Neural networks run on numbers, and DSA gives us ways to store and move numbers.

Arrays and matrices

- An **array** is an ordered list of values. The feature list `[0.8, 0.6]` is an array.
- A **matrix** is a grid of values arranged in rows and columns. A tiny gray-scale image *is* a matrix, because each cell holds a brightness number.

```
A 3x3 gray-scale image (invented brightness values, 0 = black, 255 = white)

  [  12  200  198 ]
  [  15  210  205 ]    -> can be "flattened" into an array of 9 numbers
  [  10   14   11 ]       [12, 200, 198, 15, 210, 205, 10, 14, 11]
```

Keeping the input sizes consistent

The **nine numbers** in the 3-by-3 pixel example above illustrate one way to represent a tiny image. Our earlier neuron calculation uses a different, simplified representation: **two pre-measured features**—boxiness (`0.8`) and relative size (`0.6`). The chapter assumes those two feature scores have already been provided; it does not teach an image-processing method for extracting them from pixels.

Network weights can be stored in a **matrix**. In our two-feature example, the hidden layer has two neurons, each with two incoming weights, so the weights form a **2-by-2 matrix**:

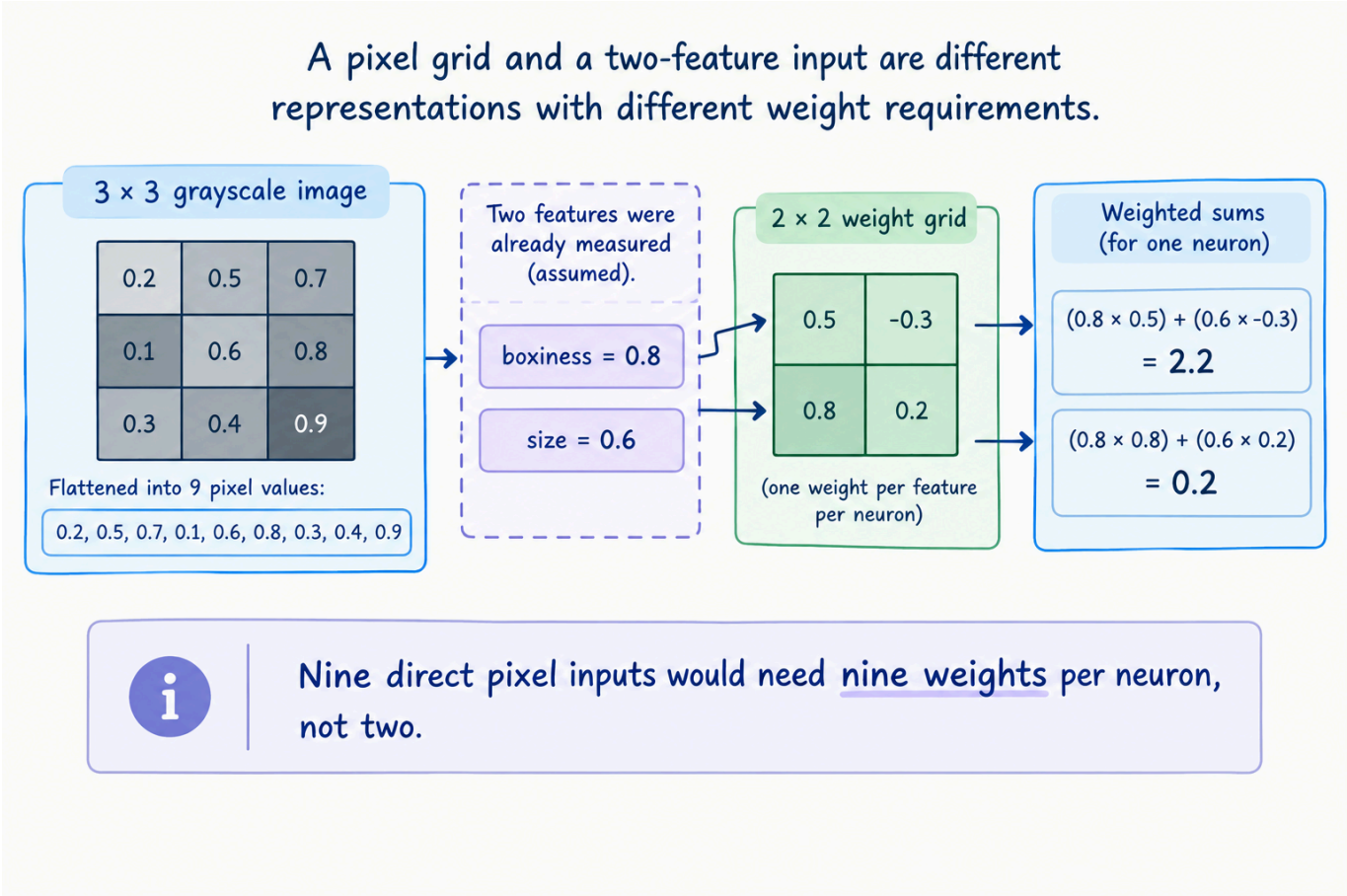
```
Two input features:                 [0.8, 0.6]     (2 values)

Weights for two hidden neurons:    [ 2,  1 ]      (2 rows x 2 columns)
                                   [ 1, -1 ]

Hidden A sum: (2 x 0.8) + (1 x 0.6)  = 2.2
Hidden B sum: (1 x 0.8) - (1 x 0.6)  = 0.2
                                      [2.2, 0.2]  (2 sums)
```

Why sizes matter: Each neuron needs one weight per input. A model taking the **nine pixel values directly** would need **nine weights per neuron**, not two; two such neurons would use a 2-by-9 weight matrix. Our 2-by-2 example only applies *after* we switch to the two-feature representation. That distinction keeps the arithmetic honest without requiring advanced matrix algebra.

Doing these weighted sums in an organized group is one reason arrays and matrices are useful. Practical neural networks can store very large numbers of parameters in multidimensional arrays.



What the earlier DSA tools do and do not do

> *Cross-reference note for editors: align the wording below with the final DSA chapter headings.*

DSA tool	Useful in a Harbor-17 workflow for...	Does it classify images?
Arrays / matrices	Storing pixel values, feature lists, weights	Supplies the numbers the network computes on
Hash table	Retrieving an image record by its ID, or looking up a label	**No**
Graph	Showing relationships, such as which cameras, locations and images are linked	**No**
BFS / DFS	Exploring that graph, e.g., finding images connected to a location	**No**

A hash table can retrieve a record such as **V-02** quickly by exact ID. The **neural network**, rather than the hash table, performs the image classification in our example. Other classification methods exist, but the distinction is the same: retrieving a record is not the same as interpreting its visual content. **Fictional analyst workflow: organize → search → predict → verify.** Each step answers a different question; none turns a model score into proof.

Worked connection: one image through the whole workflow

Follow a fictional record called **D-01** through the system. This short example combines the DSA and Connectionist AI ideas without claiming that one piece of software does every job.

Step	What happens	Relevant idea
1. Retrieve	Look up image ID `D-01` in the record system.	Hash table: exact-key retrieval
2. Represent	Supply two invented features: `[0.8, 0.6]`.	Array: ordered numerical inputs
3. Calculate	Hidden-neuron sums become `2.2` and `0.2`, giving outputs `1` and `0`.	Weights, activation and hidden layers
4. Predict	The final neuron produces `1` ("truck-like").	Inference with fixed parameters
5. Verify	A fictional reviewer checks the source image and independently records the category.	Model output is not proof

The toy network in this table produces a binary `0` or `1`. It **does not** produce the separate, fictional confidence scores used in Activity B.

> **Check yourself.** Can BFS tell us whether a photo contains a truck? **Answer:** No. BFS explores connected data. It can help find *which* images to examine, but it does not interpret image content.

---

7. Prediction Errors and Limitations

A classifier makes two kinds of mistakes. Treat "truck" as the **positive** answer:

	Actually a truck	**Actually not a truck**
Predicted truck	Correct (true positive)	**False positive** (a van called a truck)
Predicted not truck	**False negative** (a truck missed)	Correct (true negative)

- A **false positive** is a prediction of "yes" when the truth is "no". It wastes an analyst's time and may point toward the wrong thing.
- A **false negative** is a prediction of "no" when the truth is "yes". The important image may never be looked at.

Both are possible in the same system. **Reducing one can increase the other**, for example when a classifier raises or lowers its decision threshold. No threshold guarantees a correct prediction every time.

Why errors happen: unrepresentative training data

A network learns from its training examples and may generalize to new ones, but its reliability can drop when new images differ substantially from what it has seen. Suppose the Harbor-17 classifier is trained **only on daytime images from one fixed camera facing a loading dock**. When it meets new images, such as **night-time photographs, rain, or a different camera angle**, it performs poorly. Maybe it calls dark, blurry trucks "not truck", producing false negatives, because it never learned what those look like. The model may be functioning as programmed; the training examples simply did not represent the full range of conditions in which it is being used.

The same principle matters for fairness. If a training set over-represents some situations or groups, the model may work well for them and badly for others, and its overall accuracy figure can hide this. Good evaluation checks performance *separately* across different kinds of cases.

> **Two meanings of "bias": do not confuse them.**
>
> - **Bias in training data:** the examples are skewed or unrepresentative, so predictions are systematically worse in some situations. This is a data and fairness problem.
> - **The bias parameter:** a number added to a neuron's sum (equivalent to moving the threshold, Section 3). This is ordinary arithmetic, not unfairness.

Confidence scores are not guarantees

Many classifiers output a **confidence score**, such as 0.88. It is tempting to read that as "88% chance this is a truck". Do not. A score is the model's internal output. It is **not** automatically a well-calibrated probability: without evidence of calibration, `0.88` should not be interpreted as an 88% real-world chance that the prediction is correct. Scores can be high even when a model is wrong, especially on images unlike its training data.

Why independent verification is essential

Roughly speaking, a prediction means the model detected patterns it associates with the "truck" label. It does not independently establish what the vehicle is, and it does not confirm anything about an event. Treat the

output as a **lead** to check, not proof.

How to verify a lead (fictional, general checklist):

- ☐ **Original source:** Look at the original image and confirm which record the score refers to.
- ☐ **Metadata and time:** Check any available date, time, or image metadata; note that metadata can be incomplete or unreliable.
- ☐ **Independent corroboration:** Look for other information that supports or challenges the interpretation.
- ☐ **Human review:** Have a person inspect the image and record remaining uncertainty rather than relying on the score alone.

> **Check yourself.** A classifier trained only on daytime images misses a truck in a night photo. Is this a false positive or a false negative, and why did it happen? **Answer:** A false negative (an actual truck was predicted "not truck"). It happened because the training data did not represent night conditions.

---

8. Activity B: Evaluate an AI Lead

**Scenario ** In Harbor-17, an invented classifier flagged some roadside images as "truck". A supervisor asks you to evaluate eight results. For this exercise, a test set of images has been labeled by trained reviewers, so the true category is known. All data below is invented.

Image ID	Model prediction	Score for predicted class	Verified actual
V-01	truck	0.91	truck
V-02	truck	0.88	delivery van
V-03	not truck	0.79	car
V-04	not truck	0.65	truck
V-05	truck	0.72	truck
V-06	not truck	0.93	car
V-07	truck	0.55	car
V-08	not truck	0.60	car

Inputs to the model were numerical features extracted from each image. *Predicted outputs* are the "Model prediction" and score columns. Each score belongs to the **class that the model predicted** (including "not truck"); these are fictional scores, not validated probabilities. "Verified actual" is the exercise's assumed reference label, based on fictional human review.

Worked example: image V-02

1. **Read the lead.** The model says "truck" with score 0.88.
2. **Compare with verified information.** Reviewers recorded the vehicle as a **delivery van**. Treat "truck" as positive, so a van counts as "not a truck".
3. **Name the result.** Predicted truck, actually not truck: a **false positive**.

4. **Interpret the score.** The score of 0.88 was high, yet the prediction was wrong. High scores do not guarantee correctness.
5. **Decide what the lead is worth.** It is not evidence that a truck was near the warehouse. It is a prompt to look at the image.

A plausible cause: boxy rear and large size are exactly the features our neurons weighted heavily (Section 3), and vans share them.

Practice questions

Q1. Classify each of V-01, V-04 and V-07 as true positive, false positive, true negative or false negative.

Q2. Count the true positives, false positives, true negatives and false negatives across all eight images.

Q3. What fraction of the eight predictions were correct?

Q4. V-06 has a score of 0.93 for its predicted class, "not truck". An analyst says, "So there's a 93% chance it's not a truck, and we can ignore it." Explain what is wrong with this reasoning.

Q5. Suppose the training images were all daytime and all but one of the test images in this table are daytime. What could go wrong when the classifier is used on night-time images, and which error type would you watch for?

Q6. The classifier flags V-05 (a true positive). Explain why this still does not establish that a truck was involved in any real-world event.

Optional Q7 — Precision, in plain language. Of the images the model **flagged as trucks**, how many really were trucks? What fraction is that? (This is called **precision**.)

Optional Q8 — Recall, in plain language. Of the images that **actually were trucks**, how many did the model find? What fraction is that? (This is called **recall**.)

Answer key

A1. V-01: predicted truck, actually truck = **true positive**. V-04: predicted not truck, actually truck = **false negative**. V-07: predicted truck, actually car = **false positive**.

A2.

Result	Images	Count
True positive	V-01, V-05	2
False positive	V-02, V-07	2
True negative	V-03, V-06, V-08	3
False negative	V-04	1

The total is 2 + 2 + 3 + 1 = 8, as expected.

A3. Correct predictions = 2 true positives + 3 true negatives = 5 of 8, or **62.5% accuracy**. That is the fraction of all eight predictions that matched the reference labels.

A4. The score is the model's internal output, not a verified or calibrated probability, so "93%" cannot be read as a real-world chance. The test table also shows high scores are not a guarantee (V-02 scored 0.88 and was wrong). The image should be checked, not ignored. In this table V-06 happens to be correct, but we only know that because it was verified.

A5. The model has little experience of night conditions, so its predictions there may be unreliable. Expect more **false negatives** (trucks missed in dark, blurry images) and possibly more false positives too. The table's mostly daytime test set would *hide* this problem, so the 62.5% figure may overstate real performance on night images.

A6. The prediction addresses the image's appearance, not who was driving or whether a vehicle was involved in an event. The original source, time, metadata and independent corroborating information still need review. A "truck" prediction is at most a reason to look closer.

Optional A7 — Precision. The model flagged **4 images** as trucks: V-01, V-02, V-05 and V-07. **2 of those 4** really were trucks (V-01 and V-05). Therefore **precision = 50%**: half of what it flagged was correct.

Optional A8 — Recall. There were **3 actual trucks** (V-01, V-04 and V-05). The model found **2 of those 3** (V-01 and V-05). Therefore **recall = 2/3 ≈ 66.7%**: it found about two-thirds of the trucks that were present.

---

9. End-of-Chapter Review

Summary

- **Connectionist AI** learns patterns using many connected artificial neurons, with knowledge stored in numerical **weights**.
- An **artificial neuron** computes a weighted sum of its inputs and applies an **activation** to produce an output. It is a loose, simplified inspiration from biology.
- **Layers** (input, hidden, output) let a network combine features step by step. A **forward pass** moves data from input to output. Hidden neurons' neat labels in our toy example are illustrations, not literal rules.
- **Training** adjusts weights using labeled examples and error feedback. **Inference** uses the fixed learned weights on new inputs.
- **Arrays and matrices** store pixel values, features and weights. The number of weights feeding each neuron must match the number of inputs it receives. Hash tables, graphs, BFS and DFS support the surrounding workflow but do not classify images.
- Models make **false positives** and **false negatives**, and unrepresentative training data makes these errors more likely.
- A prediction is a **lead**, never established evidence, until independently verified.

Glossary

Term	Plain-English meaning
Connectionist AI	AI that learns patterns using many simple, connected numerical units
Artificial neuron	A small calculation: weighted sum of inputs, then an activation
Feature / input	A number describing something about an example
Weight	A number controlling how strongly an input influences a neuron

Term	Plain-English meaning
Activation	The rule that turns a neuron's sum into its output
Bias parameter	A number added to a neuron's sum (equivalent to shifting the threshold)
Layer	A group of neurons at the same stage of the network
Hidden layer	A layer between the input and the output
Forward pass	Moving data from input to output through the network
Label	The correct answer, supplied by a person, for a training example
Training	Adjusting model parameters using examples (labeled examples in this supervised task)
Inference	Using trained model parameters to make predictions without updating them
Generalization	Making useful predictions on new examples not used for training
Held-out examples	Examples kept out of weight updates to evaluate performance
False positive	Predicted "yes" when the truth is "no"
False negative	Predicted "no" when the truth is "yes"
Precision	Of everything the model flagged as positive, how much was actually positive?
Recall	Of everything actually positive, how much did the model successfully flag?
Bias in training data	Training examples that are skewed or unrepresentative
Confidence score	A model's output number; not automatically a true probability

Review questions

1. What is the main difference between a fixed-rule program and a neural network? A fixed-rule program uses decision rules explicitly specified by a programmer. A neural-network model learns many numerical parameter values from examples, while the algorithms and overall model design remain specified.

2. What is the difference between training and inference? During training the model parameters are adjusted using examples. During ordinary inference, the trained parameters remain fixed while the model makes a prediction.

3. In the neuron example (`w1 = 2`, `w2 = 1`, threshold 1.5), what is the output for `x1 = 0.5`, `x2 = 0.4`? (2 x 0.5) + (1 x 0.4) = 1.4, which is below 1.5, so the output is 0.

4. Give one reason a classifier might do poorly on new images even if it did well in testing. The new images may differ from the training and test data (for example, different lighting or camera angle), so the learned patterns do not apply well.

5. Why can't a "truck, 0.88" result be used as proof about a real-world event? It is a prediction about image appearance, not a verified fact. The score is not a guaranteed probability, and the model's errors, the data's limitations, and the image's origin all need independent checking.

Misconceptions to avoid

1. **"Neural networks work like human brains."** They are loosely inspired by brains and are really weighted arithmetic.
2. **"Neural networks do not use algorithms."** A network is a model trained and run using algorithms; its learned parameters do not replace the programmed computations.
3. **"The model learns from every image it sees."** Not during ordinary inference. Parameter updates require a training or learning process.
4. **"A high confidence score means the answer is correct."** Scores can be high and wrong.
5. **"Data structures like BFS or hash tables classify the images."** They organize and retrieve. The network predicts.
6. **"More data always means fairer results."** Quantity alone does not guarantee representative data or fair performance. A large but skewed dataset can still mislead.
7. **"The AI says it, so it's evidence."** A prediction is a lead until verified.

---

10. References

Introductory online resources:

- Google for Developers, *Machine Learning Crash Course*, Neural networks: Nodes and hidden layers — neurons, weights, bias, and layers.
- Google for Developers, *Machine Learning Crash Course*, Neural networks: Activation functions — how outputs are transformed.
- Google for Developers, *Machine Learning Crash Course*, Classification: Accuracy, recall, precision, and related metrics — confusion-matrix terminology and evaluation metrics.
- Google for Developers, *Machine Learning Crash Course*, Overfitting — held-out data and generalization.

Course references:

- Brookshear, J. G., and Brylow, D., *Computer Science: An Overview*.
- Russell, S., and Norvig, P., *Artificial Intelligence: A Modern Approach*.
- The project's approved module scope and final DSA chapter.